虚拟化cpu

一、进程

1.1 进程创建

1.2 进程状态

1.3 进程数据结构

二、进程API

2.1 fork()

2.2 wait()

2.3 exec()

三、机制：受限直接执行

3.1 基本技巧：直接执行

3.2 如何防止进程越权操作

3.3 如何在进程之间切换

3.3.1 协作式

3.3.2 非协作式

3.4 并发场景

四、进程调度

4.1 先进先出（FIFO）

4.2 短作业优先

4.3 最短完成时间优先

4.4 新的指标：响应时间

4.5 轮转

4.6 I/O存在

4.7 小结

五、多级反馈队列

5.1 基本规则

5.2 如何改变优先级

5.3 提高优先级

5.4 更好的计时方法

5.5 队列调优及其他问题

5.6 小结

六、多处理器调度

6.1 多处理器架构

6.2 缓存亲合度

6.3 单调度队列

6.4 多队列调度

6.5 小结

本文会梳理关于操作系统关于cpu 部分的相关知识点，目的在于对操作系统形成一个整体的认识。通过操作系统技术的发展历史，我们能够很直观的体会到操作系统存在的必要性，为什么一定要存在操作系统？他在计算机软硬件体系中属于什么角色？下面简单介绍一下操作系统的发展。

- 纸带打孔系统：我们都听说过纸带打孔技术，最原始的计算机是通过带孔的纸带记录代码指令和数据，需要有一个人将纸带输入机器，等待计算完成，将输出的纸带取走。我们可以看到目前的瓶颈在于人力，人的操作在机器看来属于时间永恒。所以下一代的计算机优化方向在于如何提高人工效率，由此出现了批处理系统。
- 批处理系统：使机器自动的处理这些纸带，这样大大提高了处理的效率，此时出现了另外一个瓶颈，在io时，cpu只能处于等待状态，人力瓶颈解决后io的速度成为了新的瓶颈。为了解决一次只能运行一个程序的问题，出现了多道程序系统
- 多道程序系统：一次可以运行多个程序，前提是该程序需要提前装载到内存中，并且能够完全装入（这与现在的多进程是完全不同的，不要混淆），这样就完成了在程序处理io时可以切换到另一个程序上运行，但是到目前为止，计算机运行起来之后是无法与人进行交互的，在性能短暂满足的时代下，催生了易用性的发展，人们需要与计算机进行交互想在运行时做一些其他操作，比如在编译时浏览代码，这就出现了分时操作系统
- 分时操作系统：通过固定时间片来分割进程运行的时间，每个进程都有机会运行，按照不同的调度策略决定在某个时间需要某个进程运行。

这些历史仅从cpu的角度来看的，其实在这些技术发展的过程中，其他的硬件也在发展，比如多道程序系统的切换，需要保存另外一个程序的现场，这就需要新增寄存器，并且内存的分布也需要调整，再如分时操作系统，将内存也进行了虚拟化，这时候催生了很多的硬件设备，比如MMU，TLB，等，这些软硬件协同配合，协同发展，才有了现在的通用型操作系统。

- 历史

早期操作系统：只是一些库

一开始，操作系统甚没有做太多事情。基本上，它只是一组常用函数库。例如，不是

让系统中的每个程序员都编写低级 I/O 处理现码，而是让“OS”提供这样的 API，这样开发

人员的工作更加轻松。

通常，在这些老的大型机系统上，一一运行一个程序，由操作员来控制。这个操作员

完成了你认为现现操作系统会做的许多事情（例如，决定运行作业的顺序）。如果你是一个

聪明的开发人员，就会对这个操作员很好，这样他们可以将你的工作移动到队列的前端。

这种计算模式被称为批（batch）处理，先把一些工作准备好，然后由操作员以“分批”

的方式运行。此时，计算机甚没有以交互的方式使用，因为这样做成本太高：让用户坐在

计算机前使用它，大部分时间它都会闲置，所以会导致设施每小时浪费数千美元[BH00]。

超越库：保护

在超越常用服务的简单库的发展过程中，操作系统在操理机器方面扮演着更为重要的

角色。其中一个重要方面是意识到现表操作系统运行的现码是特殊的。它控制了设备，因

此对待它的方式应该与对待正常应用程序现码的方式不同。为什么这样？好吧，想象一下，

假设允许任何应用程序从磁盘上的任何地方读取。因为任何程序都可以读取任何文件，所

以隐私的概念消失了。因此，将一个文件系统（file system）（操理你的文件）实现为一个库

是没有意义的。实际上，还需要别的东西。

因此，系统调用（system call）的概念诞生了，它是 Atlas 计算系统[K+61，L78]率先采

用的。不是将操作系统例程作为一个库来提供（你只需创建一个过程调用（procedure call）

来访问它们），这里的想法是添加一些特殊的硬件指令和硬件状态，让向操作系统过渡变为

更正式的、受控的过程。

系统调用和过程调用之间的关键区别在于，系统调用将控制转移（跳转）到 OS 中，同

时提高硬件特权级别（hardware privilege level）。用户应用程序以所谓的用户模式（user mode）2.6 简单历史

13

运行，这意味着硬件限制了应用程序的功能。例如，以用户模式运行的应用程序通常不能

发起对磁盘的 I/O 请求，不能访问任何物理内存页或在网络上发送数据包。在发起系统调用

时 [通常通过一个称为陷阱（trap）的特殊硬件指令]，硬件将控制转移到预先指定的陷阱处

理程序（trap handler）（即预先设置的操作系统），甚同时将特权级别提升到内核模式（kernel

mode）。在内核模式下，操作系统可以完全访问系统的硬件，因此可以执行诸如发起 I/O 请

求或为程序提供更多内存等功能。当操作系统完成请求的服务时，它通过特殊的陷阱返回

（return-from-trap）指令将控制权交还给用户，该指令返回到用户模式，同时将控制权交还

给应用程序，回到应用离开的地方。

多道程序时代

操作系统的真正兴起在大主机计算时现之后，即小型机（minicomputer）时现。像数字

设备公司（DEC）的 PDP 系列这样的经典机器，让计算机变得更加实惠。因此，不再是每

个大型组织拥有一台主机，而是组织内的一小群人可能拥有自己的计算机。毫不现现，这

种成本下降的主要影响之一是开发者活动的增加。更聪明的人接触到计算机，从而让计算

机系统做出更有趣和漂亮的事情。

特别是，由于希望更好地利用机器资源，多道程序（multiprogramming）变得很普遍。

操作系统不是一一只运行一项作业，而是将大量作业加载到内存中甚在它们之间快速切换，

从而提高 CPU 利用率。这种切换非常重要，因为 I/O 设备很慢。在处理 I/O 时让程序占着

CPU，浪费了 CPU 时间。那么，为什么不切换到操一份工作甚运行一段时间?

在 I/O 进行和任务中断时，要支持多道程序和重叠运行。这一愿望迫使操作系统创新，

沿着多个方向进行概念发展。内存保护（memory protection）等问题变得重要。我们不希望

一个程序能够访问操一个程序的内存。了解如何处理多道程序引入的甚发（concurrency）问

题也很关键。在中断存在的情况下，确保操作系统正常运行是一个很大的挑战。我们将在

本书后面研究这些问题和相关主题。

当时主要的实际进展之一是引入了 UNIX 操作系统，主要归功于贝尔实验室（电话公

司）的 Ken Thompson 和 Dennis Ritchie。UNIX 从不同的操作系统获得了许多好的想法（特

别是来自 Multics [O72]，还有一些来自 TENEX [B+72]和 Berkeley 分时系统[S+68]等系统），

但让它们更简单易用。很快，这个团队就向世界各地的人们发送含有 UNIX 源现码的磁带，

其中许多人间后参与甚添加到系统中。请参阅补充了解更多细节①。

摩登时代

除了小型计算机之外，还有一种新型机器，便宜，速度更快，而且适用于大众：今天

我们称之为个人计算机（Personal Computer，PC）。在苹果公司早期的机器（如 Apple II）

和 IBM PC 的引领下，这种新机器很快就成为计算的主导力量，因为它们的低成本让每个桌

子上都有一台机器，而不是每个工作小组共享一台小型机。

遗憾的是，对于操作系统来说，个人计算机起初现表了一一巨大的倒退，因为早期的系统忘

① 我们将使用补充和其他相关文本我，让你注意到不太适合文本主线的各种内容。有时有，我们甚至会用它们来开我我，为什

么在这个过程中没有一点乐趣？是的，许多我话都很糟糕。14

第 2 章操作系统介绍

记了（或从未知道）小型机时现的经验教训。例如，早期的操作系统，如 DOS（来自微软的磁

盘操作系统），甚不认为内存保护很重要。因此，恶意程序（或者只是一个编程不好的应用程序）

可能会在整个内存中甚写甚七八糟的东西。第一现 macOS（V9 及更早版本）采取合作的方式

进行作业调度。因此，意外陷入无限循环的线程可能会占用整个系统，从而导致重新启动。这

一现系统中遗随的操作系统功能造成的痛苦列表很长，太长了，因此无法在此进行全面的讨论。

幸运的是，经过一段时间的苦难后，小型计算机操作系统的老功能开始进入台式机。

例如，macOS X 的核心是 UNIX，包括人们期望从这样一个成熟系统中获得的所有功能。

Windows 在计算历史中同样采用了许多伟大的思想，特别是从 Windows NT 开始，这是微软

操作系统技术的一一巨大飞跃。即使在今天的手机上运行的操作系统（如 Linux），也更像

小型机在 20 世纪 70 年现运行的，而不像 20 世纪 80 年现 PC 运行的那种操作系统。很高兴

看到在操作系统开发鼎盛时期出现的好想法已经进入现现世界。更好的是，这些想法不断

发展，为用户和应用程序提供更多功能，让现现系统更加完善。

下面描述了三个主要的问题

- 操作系统通过哪些机制和策略来实现虚拟化?
- 操作系统如何有效地实现虚拟化?
- 需要哪些硬件支持?

虚拟化通俗来讲就是操作系统通过将物理资源转为通用并且易用的虚拟形式的一项技术，如：cpu的并发模型，虚拟内存的设计，等。

下面会通过代码示例来具象化的展示出操作系统虚拟化的成果

```c
#include "common.h"
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\n");
        exit(1);
    }
    char *str = argv[1];
    while (1) {
        Spin(1);
        printf("%s\n", str);
    }
    return 0;
}
```

```
编译: gcc -o cpu cpu.c -Wall
运行: ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
同时运行4个
```

我们在刚开始学习编程时大多数都操作过这种场景：同时运行多个相同的进程，并且他们似乎在同时运行。

这种现象就是cpu虚拟化的一种表现，操作系统在cpu的支持下提供了这种假象，即拥有非常多的虚拟cpu的假象。

将单个cpu转换为看似无限数量的cpu从而让许多程序看似同时运行，这就是所谓的虚拟化cpu。

一、进程

在操作系统中，进程是一个运行中的程序。在一个计算机中运行着远大于cpu核心数的进程，这时候并没有感觉到异常，反而使用起来非常简单，这时候就出现了一个关键的挑战：

> 如何提供有许多cpu的假象?

现在的cpu使用的是时分共享机制，通过使不同的进程分别运行一段时间，不停的切换进程来实现。这时候用户就能够运行多个程序，看起像同时运行一样。但是这种分享cpu的方式，必定会导致进程运行变慢(相对于独占而言)。

在这些机制(时分共享，空分共享)之上，存在一些策略，用于决定如何使用该机制，比如进程调度策略，他利用进程行为的历史信息，工作负载，进程类型，性能指标，来综合决策。

操作系统为正在运行的程序提供的抽象结构就是进程。在进程中有一个明显的组成部分，就是内存，指令在内存中，读取和写入的数据也在内存中。

另一个部分是寄存器，指令的执行会明确的操作寄存器。他们对执行进程很重要。

1.1 进程创建

操作系统在运行程序是首先会将代码和静态数据加载到内存中，也就是进程的地址空间中。这里需要说明：

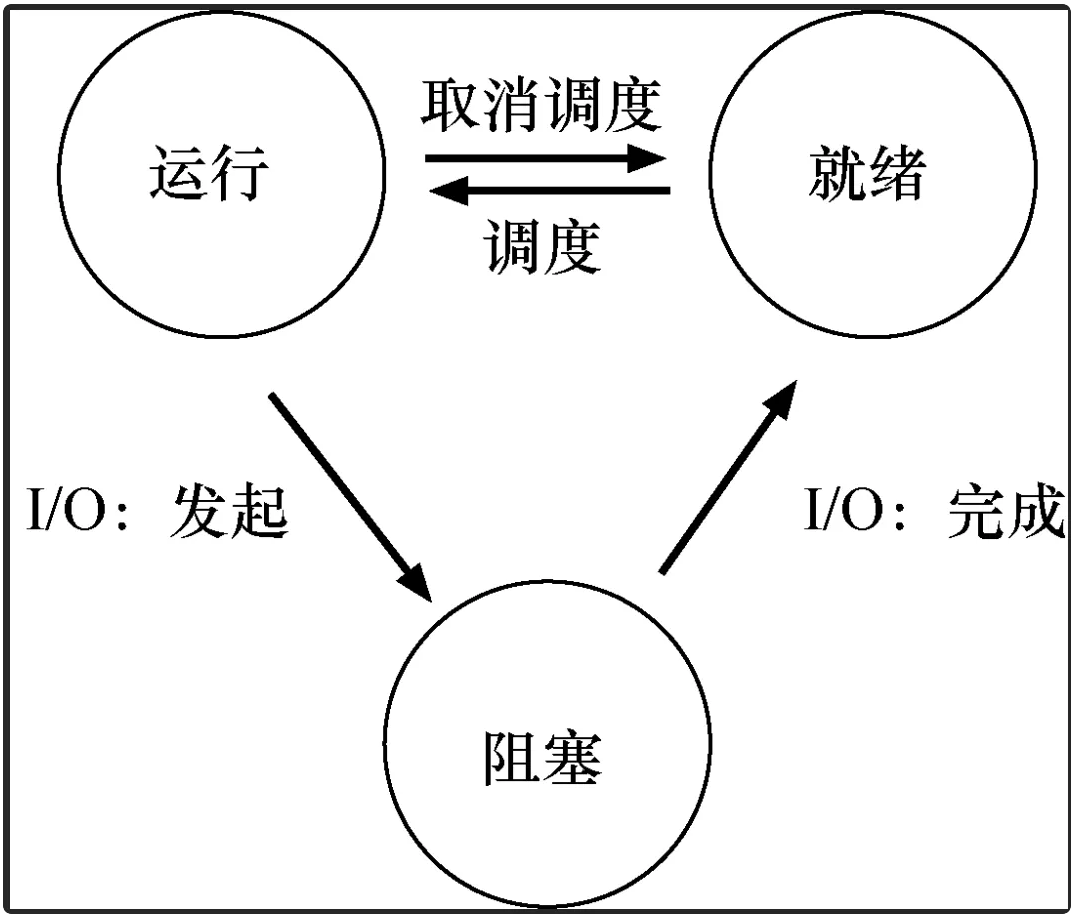
- 早期的操作系统中，如批处理那个时代的，会完全进行加载
- 现代的操作系统会在执行期间进行加载，这时候会涉及到分页和交换的机制。

代码和静态变量加载成功后，然后创建栈，堆以及设置io，和各种初始工作(这个涉及到task_struct的结构，进程创建时该结构是如何被填充的？)，最后会进到main函数处，开始执行进程，此时cpu的控制权交给了该进程。

1.2 进程状态

进程创建运行后会处于不同的状态，我们来说明一下早期的情况：

- 运行态：正在运行中，获取到了cpu的使用权
- 就绪态：进程准备好运行，等待操作系统进行调度
- 阻塞态：进程执行了某种操作，导致自身被阻塞，需要等带该操作完成才能继续执行，，如等待io



这里简单说明一下进程调度的过程，一个进程被创建出来处于就绪态，等待操作系统进行调度，这时候可能每个核心都被其他进程占用，那么操作系统是怎么能够将该进程调度起来呢？这就牵扯到一个硬件的配合机制(时钟中断)，操作系统在初始化的时候，会预置一些终端处理函数，其中包含调度的函数。当时钟中断发生时，os获得cpu的控制权，可以将其调度起来。另一中场景是进程调用read进行io操作，此时进程会将自己调度出去，变成阻塞状态，等待io的完成中断产生，此时os又获得了cpu的使用权，这时候可以决定是否将其调度起来。总之，进程的调度依赖于os获取操作系统的控制权，而获取控制权的方式有以下两种：

- 依赖时钟中断
- 依靠系统调用

当然，进入内核态的方式还有其他情况，这里只描述进程调度相关。（此处涉及内核中进程调度相关代码，linux中断机制，软中断，硬中断，如何处理中断，如何防止嵌套中断等。此处还涉及到多进程的组织形式，内核以什么数据结构进行进程的存储）

1.3 进程数据结构

os为了便于管理所有的就绪进程，会为所有的就绪进程保留一个列表，用来追踪当前进程的一些信息，当然，阻塞的进程也有对应的组织结构，当io完成时，需要正确的唤醒对应的进程(涉及到进程调度，中断的设计。io通知和进程如何关联)。

二、进程API

2.1 fork()

fork函数调用一次可以返回两个结果，背后的原理是以当前进程为基础，创建了一个新的进程，新的进程继承了园进程的一些特性(涉及到task_struct的创建，以及各个变量的含义，其中哪些变量被直接复制的，哪些变量会变更为自己的。包括页表的复制，写实时拷贝技术的实现等)。

我们要知道的是，fork的执行流程是从函数调用点之后生效的，后面的代码才会被重复执行。

2.2 wait()

当我们创建完进程后，我们需要等待他运行完毕，这样才是一个完整的流程，如果创建后不管不顾，等父进程退出的时候他就会变成孤儿进程，如果子进程先退出了，父进程没有使用wait进行善后处理，那么子进程就会变成僵尸进程（这里需要了解僵尸进程的危害，占用进程pid号，但是他的资源都已经被释放了。）我们可以通过wait的返回值获取到子进程的退出状态。

2.3 exec()

我们在fork后，创建了一个新的进程，该进程的运行流程与父进程一致，我们如果想执行新的程序，就需要调用exec函数。他会在当前的进程空间中构建一个全新的进程对象，用于执行新的程序。所以exec后面的代码是无法运行的。exec需要明确的是，他会重置信号处理函数，保留文件描述符，保留工作目录和进程id。exec会替换地址空间，代码段，数据段，该操作会提高进程执行的效率(防止进程写时拷贝导致的内存复制)。

三、机制：受限直接执行

在虚拟化cpu的时候，不同的进程需要共享物理cpu，可以通过时分共享来实现。共享cpu的方式涉及到进程的切换，我们如何能不增加系统开销的情况下去操作这些进程？并且保持系统的控制权。这就要考虑一下性能和安全性。

关键问题：如何高效、可控地虚拟化 CPU

操作系统必须以高性能的方式虚拟化 CPU，同时保持对系统的控制。为此，需要硬件和操作系统支

持。操作系统通常会明智地利用硬件支持，以便高效地实现其工作。

3.1 基本技巧：直接执行

为了使程序尽可能快的运行，操作系统开发人员将进程直接在cpu上执行，那我们怎么保证进程不做一些不合规的操作?

3.2 如何防止进程越权操作

为了安全性，我们需要硬件来支持，硬件提供不同的执行模式来区分用户模式和特权模式（这两种模式由二极管来控制，特权位未被置位时，某些二进制指令时无法通过的，电流会被阻挡，可以找一个动画说明来理解），在特权模式下，操作系统可以访问机器的全部资源，在用户模式下，某些指令无法执行，会陷入内核，内核执行完成后从陷阱返回，这些陷阱表在操作系统初始化时会被注册到内核里。

3.3 如何在进程之间切换

在之前的设计中，进程是是直接运行在cpu上的，此时操作系统没有在运行，那么他又怎么能控制进程的调度呢？这就出现了不同的设计方案，

3.3.1 协作式

该方式依赖进程主动放弃cpu操作，最初的设计考虑到进程在执行系统调用时将控制权交给os，如读取文件，或者向另一台发送消息，或者创建新进程等。当程序执行了非法操作，也会将控制权交给os。但是不是所有进程都会主动让出cpu的，有一些恶意进程可能是死循环，这时就会出现很棘手的问题

3.3.2 非协作式

关键问题：如何在没有协作的情况下获得控制权

即使进程不协作，操作系统如何获得 CPU 的控制权？操作系统可以做什么来确保流氓进程不会占

用机器?

答案很简单，在计算机系统构建的初期，人们就发现了时钟中断，时钟设备可以编程为每隔几毫秒产生一次中断。这是进程被打断，进入到预设的中断程序中，这时候os就会重新获取控制权

此处需要注意的是硬件在发生中断时，需要为当前运行的进程保存足够的状态，保证在中断结束时能够继续运行。

3.4 并发场景

当多个进程都触发了中断时，这种情况该如何处理。

操作系统会在处理中断的时候禁止中断，如果中断处理过长时间的话，这样会导致中断丢失。

四、进程调度

目前为止，我们对于底层的进程切换的逻辑应该已经清楚了，接下来会描述操作系统如何对这些进程进行调度，在中断发生时，如何决定运行哪一个进程。

4.1 先进先出（FIFO）

我们可以实现一个最基本的策略，被称为先进先出，这个方法很便于实现，但是这个策略对于任务的周转时间（任务到达队列，到运行完成需要的时间）很不友好，如果有一个很快就能结束的任务被长任务阻挡住了，这就导致周转时间变长。

4.2 短作业优先

这个策略优先运行最短的任务，其次是次短的任务，以此类推。但是该方法存在一个问题，如果短作业在长作业启动时到达，这时依然会导致等待过长时间，周转时间过长。

4.3 最短完成时间优先

以上的调度策略都是非抢占式的，如果可以抢占的话，那么在短任务到来的时候，判断剩余工作和新工作谁的剩余时间少，然后调度该工作。这样可以大大减少周转时间。

4.4 新的指标：响应时间

目前来看最短完成时间优先确实是一个很好的策略，但是用户现在想要与终端进行交互，因为分时共享的引入，导致了一个新的衡量标准，响应时间，他意味着从任务到达系统到首次运行的时间。

$$T\text{ 响应时间} = T\text{ 首次运行} - T\text{ 到达时间}$$

在这种衡量指标下，如果前面存在较短的任务，那么只能等待其执行完毕，才能轮到自身被调度，这样对于交互简直是灾难。

4.5 轮转

为了解决上述问题，引入了一个新的调度算法，轮转调度算法，他会将队列内的任务分别运行相同的时间片，然后切换到下一个，直到任务被完全执行完毕。

这是需要意识到一个问题，对于运行时间片的时长确认，如果时间片过于的短，在进行上下文切换的时候成本会及其的高。上下文切换时，不仅仅需要保存和恢复寄存器的状态，还会导致一些稳定的状态被刷新掉，比如，cpu的高速缓存，TLB缓存，分支预测器等。所以我们要合理的设计时间片。

4.6 I/O存在

当io发生时，应将cpu切换到另一进程上。

4.7 小结

以上的各种策略都建立在我们对任务属性了解足够多的情况下设计的，事实是我们对每个任务的工作长度无从得知，多以我们就要将理想化的方案与可用的机制联系起来，设计一个易于使用的策略。

五、多级反馈队列

提示：从历史中学习

多级反馈队列是用历史经验预测未来的一个典型的例子，操作系统中有很多地方采用了这种技术

（同样存在于计算机科学领域的很多其他地方，比如硬件的分支预测及缓存算法）。如果工作有明显的阶

段性行为，因此可以预测，那么这种方式会很有效。当然，必须十分小心地使用这种技术，因为它可能

出错，让系统做出比一无所知的时候更糟的决定。

5.1 基本规则

MLFQ有多个队列，每个队列有不同的优先级，一个任务在同一时刻只能存在于一个队列中

- 规则一：运行优先级高的进程
- 规则二：优先级相同，轮转运行

5.2 如何改变优先级

因为任务中有些会频繁地放弃cpu，有些不在意响应时长，所以我们需要对优先级做一个动态调整

- 任务首先会放在最高优先级队列
- 用完一个时间片，降低一个优先级
- 如果在一个时间片内，主动放弃cpu，则优先级不变

这三个规则暂时看来还不错，但是有一种场景，如果一个进程总是进行io读取，并且每次都无法完成一个时间片，这就会导致一个问题，他的优先级不会被改变，进而会一直占用高优先级队列，这时之前用完一个时间片的进程，优先级被调低，再也没机会被调度了。这样看来很不合理。

5.3 提高优先级

针对于上述的问题，我们尝试新增规则来避免

- 经过一段时间S，就将系统中的所有工作重新加入最高优先级队列

这样就解决了两个问题，1、防止了进程饿死。2、如果进程从cpu密集型转为交互性，优先级提升时，调度程序会正确的对待他，而不是一直处于某低优先级的队列中

这样的设计引入了一个头疼的问题，又需要权衡一个数值来作为标准了，这个S应该设置为多少确实不太好把握。

5.4 更好的计时方法

现在的问题点是如何防止调度程序被愚弄(进程不停的放弃cpu，只为保住自己的优先级)。这里给出了一个解决方案。修改计时方案，记录每个进程在某一层的总消耗时间，而不是每次调度时重新计时，如果某个进程在该层把自己的配额用尽，则被调度到下一层中。因此我们修改规则：

- 一旦进程用完了在某一层的配额，不论中间放弃了多少次cpu，就将其降低优先级。

5.5 队列调优及其他问题

关于堆积反馈队列的算法还有一些问题，如之前所说，队列的配置，每层时间片的配置，提升优先级的间隔配置，这些还没有明确的指标说明。只能通过对于工作负载的经验，以及实践的反馈来进行调整，不断进行优化。

大多数的MLFQ变体都支持不同队列可变的时间片长度，高级队列时间片较短，低级队列大部分是计算密集型，所以时间片更长

5.6 小结

规则 1：如果 A 的优先级 > B 的优先级，运行 A（不运行 B）。

规则 2：如果 A 的优先级 = B 的优先级，轮转运行 A 和 B。

规则 3：工作进入系统时，放在最高优先级（最上层队列）。

规则 4：一旦工作用完了其在某一层中的时间配额（无论中间主动放弃了多少次 CPU），就降低其优先级（移入低一级队列）。

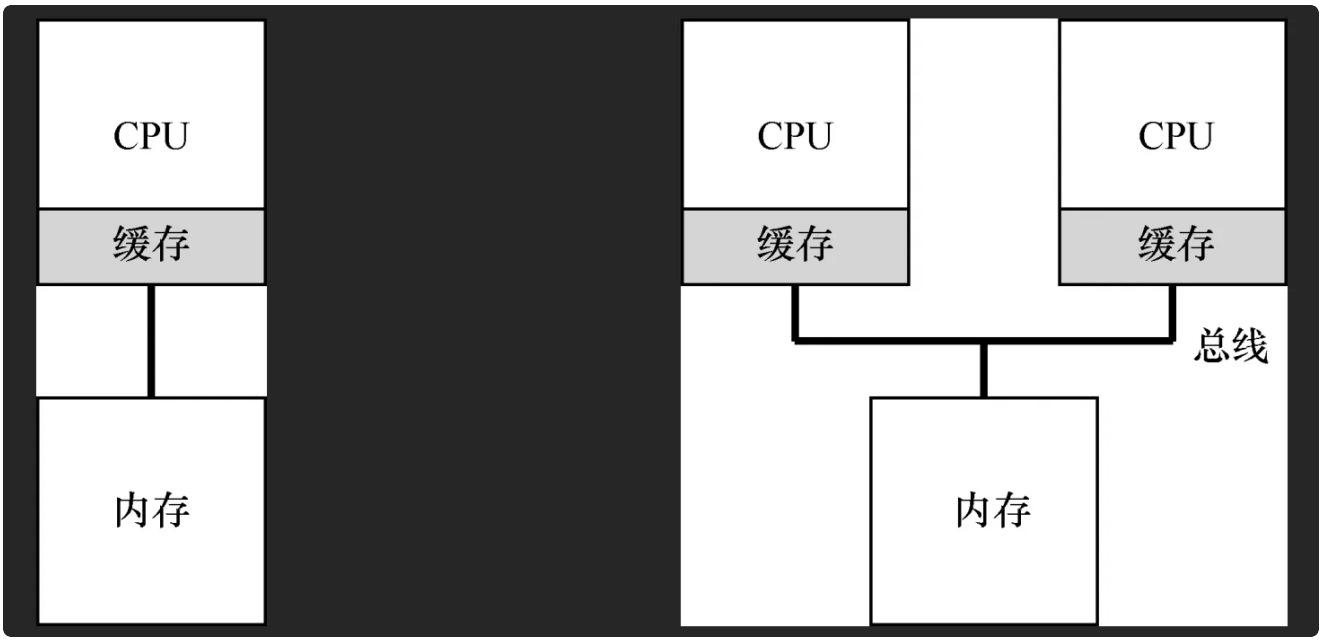
规则 5：经过一段时间 S，就将系统中所有工作重新加入最高优先级队列。

尽管多级反馈队列看着很优秀，但是目前已经被CFS调度算法所取代，在目前的Linux内核中，已经使用了全新的调度算法。

六、多处理器调度

6.1 多处理器架构

首先，我们应该了解多处理器和单处理器的基本区别。主要区别在于对硬件缓存的使用，以及多处理器之间共享数据的方式。(这里在学习的时候，误以为多处理器导致了缓存的可见性问题，这里一定要明白需要屏障的原因是什么？防止将多线程作为屏障的催生原因。所以这里不讨论屏障的问题，后面并发的章节会单独分析屏障问题。)



在单cpu系统中，存在多级的硬件缓存，当cpu需要从内存中加载指令并且读取一个值时，第一次读取会从内存中获取，花费较长时间，然后处理器将其放入缓存中，下次在读取就会在缓存中找到该数据，大大加快了运行速度，这里涉及到时间局部性和空间局部性

在多cpu的情况下，缓存变得复杂得多。例如，当cpu1读取了内存的数据A，然后修改为A1，此时在缓存中还未更新到内存中，这时候中断发生，将其调度到了cpu2上，这时候会重新读取该值，但是内存中还是旧的A，这就导致了严重的问题。这种普遍的问题称为缓存一致性问题。

硬件提供了缓存一致性问题的基本解决方案：通过总线嗅探方案，来监控内存访问，保证内存的唯一性（缓存一致性可以详细了解一下，设计思想对于上层应用也有很好的启发。）。

6.2 缓存亲合度

一个进程在某个cpu上运行时，会在该cpu中维护很多常用的状态，如果调度到不同的cpu上会导致状态被废弃，重新加载数据，所以在调度时最好保持在同一个cpu上进行切换

6.3 单调度队列

该方式存在一个很大的锁竞争导致的性能损耗，因为所有的核心都访问同一个队列，导致时间花费在获取锁上，另一个问题是，亲合度的问题，单队列导致每个进程会在不同的cpu上流转，导致维护的状态不停的被作废。

6.4 多队列调度

该调度模型中每个cpu有一个队列，这就大大减少了锁的竞争，和亲合度的问题，但是这种模型引入一个新问题：负载不均

如果cpuA的进程调度完了，此时会空闲下来，这时候虽然cpuB队列中还有任务，cpuA也不会去执行，这就导致cpu利用率下降。

应对的方法就是任务的迁移，当前队列任务为空时，将任务从其他队列中迁移过来，迁移的方式有很多种，但是窃取的策略会引入一些难以决策的问题，比如多久观察一次其他队列，一次窃取多少任务过来等，这些限制的设置依然很困难。

6.5 小结

我们应该承认设计一个完美的调度算法很困难，需要考虑扩展性，负载均衡，缓存亲合度，等等。